

- 1) Raw audio is basically just a long list of amplitude values over time. While the model could technically learn from that, it is not the most useful format because the important speech information is spread across frequency changes, timing, and sound energy. Spectrogram and MFCC-style features make the audio easier to work with by showing what frequencies are present at different points in time. This is helpful because words like “yes” and “no” are usually easier to tell apart by looking at their frequency patterns instead of the raw waveform. These features also shrink the amount of data while keeping the important speech information, which makes the model more realistic to run on a small device like the Arduino Nano 33 BLE.
- 2) The representative dataset gives the TensorFlow Lite converter a small set of realistic examples so it knows what kind of input values the model will actually see. During full integer quantization, the converter has to map floating-point values into 8-bit integer values. To do that well, it needs to estimate good ranges for the model’s tensors, such as the scale and zero-point values. In the notebook, this data is generated using `audio_processor.get_data(...)`, flattened, and then passed into the converter. Without a representative dataset, the INT8 model could choose poor ranges, which could hurt accuracy. This step is important because INT8 models use much less precision than float models, so the conversion needs to be calibrated using real-looking input data.
- 3) `audio_processor.get_data()` takes audio examples from the dataset and prepares them in the format the model expects. It can pull data from different dataset splits, such as training, validation, or testing. It also applies the same preprocessing used during training, such as time shifting, adding background noise, and converting the audio into the final feature representation. The function returns both the processed data and the matching labels. In this notebook, it is used to grab one processed testing example at a time so the representative dataset can give the quantizer realistic keyword-spotting inputs.
- 4) In the final export step, the .tflite model is turned into a C/C++ byte array so it can be included directly in the Arduino program. `g_model` is the actual model stored as bytes, and `g_model_len` is the size of that model in bytes. These need to be copied correctly because TensorFlow Lite for Microcontrollers loads the model directly from that array. If `g_model` is copied wrong, the model data can become corrupted. If `g_model_len` is wrong, the Arduino may read too much or too little data, which can cause loading errors, bad predictions, or crashes. Basically, `g_model` is the model itself, and `g_model_len` tells the Arduino how big that model is.

```
Heard unknown (203) @307248ms
Heard no (208) @308992ms
Heard unknown (218) @310928ms
Heard yes (212) @312208ms
Heard unknown (201) @315168ms
```

5)